Accelerating Suffix Jailbreak attacks with Prefix-Shared KV-cache

Anonymous Authors[1]

Abstract

Suffix jailbreak attacks serve as a systematic method for red-teaming Large Language Models (LLMs) but suffer from prohibitive computational costs, as a large number of candidate suffixes need to be evaluated before identifying a jailbreak suffix. This paper presents Prefix-Shared KV Cache (PSKV), a plug-and-play inference optimization technique tailored for jailbreak suffix generation. Our method is motivated by a key observation that when performing suffix jailbreaking, while a large number of candidate prompts need to be evaluated, they share the same targeted harmful instruction as the prefix. Therefore, instead of performing redundant inference on the duplicated prefix, PSKV maintains a single KV cache for this prefix and shares it with every candidate prompt, enabling the parallel inference of diverse suffixes with minimal memory overhead. This design enables more aggressive batching strategies that would otherwise be limited by memory constraints. Extensive experiments on six widely used suffix attacks across five widely deployed LLMs demonstrate that PSKV reduces inference time by 40% and peak memory usage by 50%, while maintaining the original Attack Success Rate (ASR). The code has been submitted and will be released publicly.

1. Introduction

Suffix jailbreak attacks, which aim to synthesize jailbreak suffixes to bypass safety guardrails and elicit harmful content, have become a vital component for red-teaming Large Language Models (LLMs) (Zou et al., 2023; Chao et al., 2024). Current approaches primarily employ heuristic search (Zou et al., 2023; Zhu et al., 2023) or LLM-based generators (Paulus et al., 2024) to generate these suffixes. However, a major bottleneck of these methods is their prohibitively high computational cost. Standard optimization-based attacks must iteratively evaluate a vast number of *candidate* suffixes against the target LLM to identify effective jailbreaks. While LLM-based generators offer faster test-time suffix generations, they merely shift the computational burden to the training phase, which still necessitates time-consuming suffix generation to construct training data.

Ideally, Key-Value (KV) caching could accelerate this evaluation process by reusing intermediate tensors. However, in jailbreak scenarios, the system prompt is typically far longer than the suffix. Standard KV cache implementations must duplicate this long prompt's cache for every candidate suffix in a batch, inducing excessive memory consumption that limits batch size. While state-of-the-art inference systems like vLLM (Kwon et al., 2023b) and SGLang (Zheng et al., 2024) address these memory constraints through careful memory scheduling, they are designed strictly for forward-pass inference. Their highly encapsulated interfaces and lack of backpropagation kernels prevent access to gradients, restricting their utility to a subset of gradient-free attacks (*e.g.*, BEAST attack (Sadasivan et al., 2024)). Consequently, these limitations collectively impede the effective acceleration of general suffix jailbreak attacks.

To overcome these limitations, we propose **Prefix-Shared KV Cache (PSKV)**, a lightweight, workload-aware, and plug-and-play framework tailored for accelerating suffix jailbreak attacks. Our method is motivated by a key *structural redundancy* in jailbreak optimization: while a batch of candidate suffixes varies, they typically share an identical targeted harmful instruction as the prefix. Standard KV cache implementations ignore this property, redundantly duplicating the long prefix cache for every candidate. In contrast, PSKV maintains a single, shared KV cache for the invariant prefix and broadcasts it to all candidate suffixes during inference. By employing a suffix-centric alignment strategy, PSKV supports dynamic expansion and parallel batched loss computation. Crucially, unlike encapsulated inference engines, PSKV integrates seamlessly into existing attack pipelines, ensuring substantial acceleration and minimal memory overhead while preserving gradient access.

We evaluated PSKV across six widely used suffix jailbreak attacks and five popular open-source models. We compared our framework against a baseline without KV cache and

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

a standard KV cache implementation. The results demonstrate that PSKV reduces inference time by approximately 40% compared to the no-cache baseline and decreases peak memory usage by 50% compared to the standard KV cache implementation. Furthermore, because PSKV is a drop-in optimization that does not alter the high-level logic of the attack, it maintains the original Attack Success Rate (ASR). This efficiency enables researchers to scale evaluations to larger models and broader search spaces that were previously computationally prohibitive.

In summary, our contributions are:

- We identify the inefficiency of general-purpose inference caching when applied to the specific structural patterns of jailbreak suffix optimization.
- We propose PSKV, a memory-efficient framework featuring virtual cache broadcasting and suffix-centric tensor alignment to eliminate redundant computations.
- We conduct comprehensive evaluations demonstrating that PSKV substantially reduces both computational and memory costs, paving the way for a scalable LLM vulnerability assessment.

2. Related Work

Automated Suffix Jailbreaking. Recent research has transitioned from manual red-teaming to automated adversarial optimization. The foundational method, GCG (Zou et al., 2023), employs a greedy, gradient-guided search to iteratively refine an adversarial suffix. Building on this, subsequent works have explored more sophisticated search strategies. BEAST (Sadasivan et al., 2024) replaces the greedy algorithm with beam search to explore a wider range of promising candidates. Zhu's AutoDAN (Zhu et al., 2023) utilizes genetic algorithms to evolve suffixes, mitigating the local optima problem of purely greedy methods. Other variants like GCQ (Gradient-based Attack with Quantization) (Hayase et al., 2024) focus on improving the efficiency of the core gradient-based update step itself.

An alternative paradigm uses LLMs themselves as generators. AdvPrompter (Paulus et al., 2024) fine-tunes a language model to automatically compose adversarial prompts. Similarly, AmpleGCG trains a generative model to directly output effective suffixes, amortizing the search cost into a one-time training process (Liao & Sun, 2024). Although these methods differ, they all require evaluating a large number of candidate suffixes to create a training set during either the model training or data generation phases. Our work presents an orthogonal optimization that accelerates this critical evaluation step, benefiting both categories of attack.

Efficient Inference & KV Caching. Optimizing Key-Value (KV) cache management is central to high-throughput LLM serving. vLLM (Kwon et al., 2023a) introduced PagedAttention to mitigate memory fragmentation in multi-user scenarios, while SGLang (Zheng et al., 2024) employs RadixAttention to automatically detect and share prefixes across diverse incoming requests via a radix tree structure. ChunkAttention (Ye et al., 2024) further optimizes memory locality. However, these systems are architected for *asynchronous, multi-user serving*, prioritizing request scheduling and fairness over raw batch latency. Applying them to the *synchronous, tight-loop* nature of adversarial attacks introduces unnecessary overhead (e.g., complex tree lookups, scheduler latency). In contrast, PSKV is a workload-specific optimization that hard-codes the "one-prefix-many-suffixes" pattern directly into the inference kernel, bypassing the complexity of general-purpose serving engines to achieve maximum efficiency for attack loops.

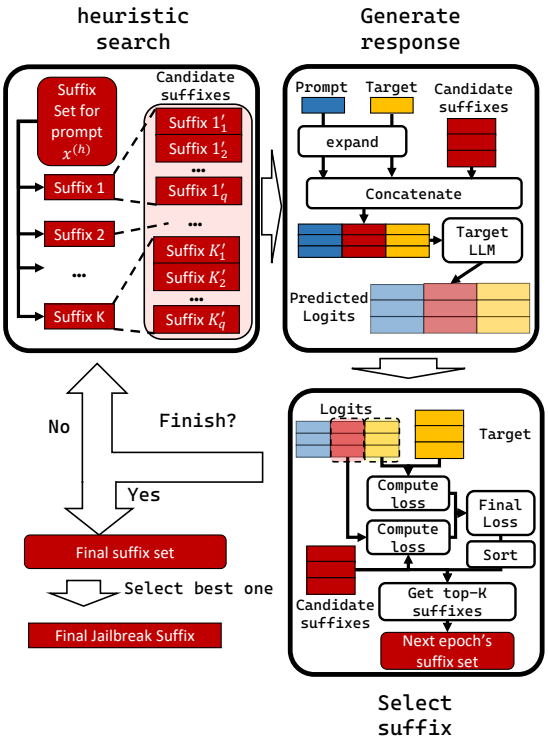

Figure 1. The computational pipeline of iterative suffix attacks.

3. Preliminaries

As illustrated in Fig 1, the workflow of a typical suffix-based jailbreak attack operates as an iterative cycle comprising three distinct phases: heuristic candidate generation, batched forward pass evaluation, and candidate selection. This pipeline forms the operational basis for the optimization problem defined below.

Adversarial Suffix Optimization. We consider an LLM as a conditional distribution $p_\theta(y|x)$ over a vocabulary $\mathcal{V}$. Given a harmful query $x^{(h)}$ (e.g., "How to build a bomb")

and a target response $y^{(h)}$, the goal of a suffix attack is to find an adversarial sequence $x^{(s)} \in \mathcal{V}^L$ that maximizes the target probability:

$$x^{(s)*} = \arg\max_{x^{(s)} \in \mathcal{V}^L} \log p_\theta(y^{(h)} | x^{(h)} \oplus x^{(s)}), \quad (1)$$

where $\oplus$ denotes concatenation. Solving Eq. (1) typically involves iterative discrete optimization. In the *heuristic search* phase, algorithms mutate the current suffix to generate a batch of N candidates, $\mathcal{S} = \{s_1, \ldots, s_N\}$. Subsequently, during the *response generation* phase, these candidates are concatenated with the static prompt and the target response to form the batch $\{x^{(h)} \oplus s_i \oplus y^{(h)}\}_{i=1}^N$. This batch is then fed into the model to compute the prediction logits for the target sequence. Based on the computed target and perplexity losses, the top-k candidates are *selected* in the suffix selection phase to seed the next iteration. Crucially, this cycle repeats for hundreds of epochs, meaning the static prefix $x^{(h)}$ is re-evaluated N times per step, creating a massive computational workload.

Standard Attention and KV Cache. The core operator in Transformer-based LLMs (Vaswani et al., 2023) is the scaled dot-product attention. For a specific layer and attention head, given a query q_t, and the key-value pairs for the current sequence history $(K_{<t}, V_{<t})$, the attention output o_t is computed as:

$$o_t = \text{Softmax}\left(\frac{q_t[K_{<t}; k_t]^\top}{\sqrt{d}}\right)[V_{<t}; v_t], \quad (2)$$

where $[\cdot;\cdot]$ denotes concatenation along the sequence length dimension. To avoid recomputing $K_{<t}$ and $V_{<t}$ at every generation step, inference systems employ KV Caching to store these tensors in GPU memory. Since standard deep learning frameworks require the KV cache batch dimension to align with the input batch size N, the KV states for the static prefix $x^{(h)}$ must be physically duplicated N times:

$$\mathcal{M}_{prefix} = \text{Repeat}(K^{(h)}, V^{(h)}, \text{times} = N). \quad (3)$$

This results in a memory complexity of $\mathcal{O}(N \cdot L_{prefix})$, which strictly limits the search width N and prevents the use of large batch sizes.

4. Methodology

In this section, we first illustrate the redundant computation problem of prefix KV vectors in suffix jailbreaking and then introduce our *PSKV* method to tackle the raised challenge and substantially improve the efficiency of suffix jailbreak attacks.

4.1. Redundant Prefix computation in Suffix Jailbreak Attacks

According to the general jailbreak suffix synthesis process described in Section 3, the main computational burden of suffix jailbreaking arises from performing $K \cdot q$ inference passes (in the response generation phase) in each synthesis iteration, where K is the number of candidate suffixes in each iteration, and q is the number of new suffixes generated for each candidate. Larger K and q usually lead jailbreak attacks to explore more candidate suffixes, thus increasing the chance of finding an effective jailbreak suffix; however, they also result in more LLM inference times.

The most straightforward implementation, which we term the **unaccelerated traditional attack method**, treats each of the $K \cdot q$ candidate prompts as independent inputs in every iteration. This approach repeatedly re-computes the KV vectors for the prefix $x^{(h)}$, leading to significant redundant computational overhead. As a result, when an LLM is performing inference on these prompts in parallel, it will redundantly calculate the same KV vectors for the same prefix $x^{(h)}$ for up to $K \cdot q$ times at each step.

A more advanced baseline would employ a **standard KV cache** implementation. The KV vectors for the prefix $x^{(h)}$ are computed only once before the iterative synthesis process begins. However, to evaluate all $K \cdot q$ suffix candidates in a single parallel batch, this pre-computed prefix cache must be duplicated $K \cdot q$ times to match the batch dimension of the suffixes, which introduces a severe memory bottleneck. For a batch of B prompts, this problem is exacerbated, requiring $B \cdot K \cdot q$ copies of the caches.

Furthermore, as shown in Table 1, the average token length of instructions in HarmBench is 78.4 (Mazeika et al., 2024), which is significantly larger than the standard jailbreak suffix length in practice (typically set to 20 tokens). Collectively, these observations suggest that if the KV vectors of the prefix harmful instruction $x^{(h)}$ can be calculated only once and used for the inference of different candidate prompts, then both the *computation* and *memory* costs of suffix jailbreaking can be considerably reduced, as a large amount of redundant intermediate KV vectors no longer need to be calculated.

Fortunately, we observe that while the number of candidate prompts that an LLM needs to perform inference on in each jailbreaking iteration is as large as $K \cdot q$, these prompts all share the same prefix, *i.e.*, the original harmful instruction $x^{(h)}$. Specifically, as illustrated in Fig. 1, the suffix jailbreak attack only modifies the suffixes of these candidate prompts but keeps the harmful instruction $x^{(h)}$ as their prefix intact.

Table 1. The average length (in characters) of harmful prompts and targeted texts in HarmBench.

Dataset	Harmful Prompt	Targeted Text
HarmBench	78.4	93.3

4.2. PSKV for Single Jailbreak Suffixes Synthesis

These observations highlight a clear opportunity for optimization. Based on our previous analysis, we now propose *PSKV*, a method that can substantially accelerate existing suffix jailbreak attacks by reducing redundant computation of prefix KV vectors during the candidate prompt inference process of suffix jailbreaking.

The overall procedures are also illustrated in Figure 2. PSKV adopts a lazy, layer-wise expansion strategy. Unlike standard implementations that pre-allocate expanded memory for all layers during initialization, we maintain a compact, single-batch storage of the prefix. Specifically, before a suffix jailbreak attack starts the iterative suffix searching process in Figure. 1, PSKV runs a single forward pass for the harmful instruction $x^{(h)}$ to compute and cache its KV vectors across all attention layers of the targeted LLM. Then, when the targeted LLM is performing inference on the $K \cdot q$ candidate prompts during the iterative suffix searching process, whenever the LLM needs to use the KV vectors of the prefix from an attention layer, PSKV directly retrieves the corresponding KV vectors from its cache and duplicates these vectors $K \cdot q$ times for the LLM to support its inference process. Compared with vanilla suffix jailbreaking, our PSKV method reduces the time-consuming prefix KV cache computation from $K \cdot q$ times to only **a single time**, which can significantly improve the computation efficiency of suffix jailbreaking.

Furthermore, we note that the KV vector duplicating procedure in PSKV requires little memory overhead. This is because during LLM inference, the outputs of all LLM attention layers are calculated in a layer-by-layer manner, and the computation process only requires the prefix KV vectors from that specific layer. As a result, the prefix KV vector duplication procedure in PSKV is also performed in a layer-by-layer manner, and at any given time, only KV vectors from a single layer are duplicated. This ensures that at any given moment, the memory-intensive expansion is isolated to a single layer, considerably reducing peak memory overhead compared to the vanilla approach where redundant prefix repetitions persist across the entire network stack simultaneously.

Complexity analysis. Let E be the number of synthesis iterations; N_p, N_s, N_t the token lengths of the prefix, suffix, and target; and $K \cdot q$ the candidates per iteration. We compare the total computational (C) and memory (M) complexities. We provide the detailed proof in Appendix B.1.

Table 2 summarizes the complexity analysis (detailed proof in Appendix B.1). While Standard KV caching reduces computational cost, it incurs a high memory footprint of due to prefix duplication. In contrast, PSKV maintains the same computational efficiency but reduces the memory complexity, effectively removing the dependency of prefix memory on the batch size.

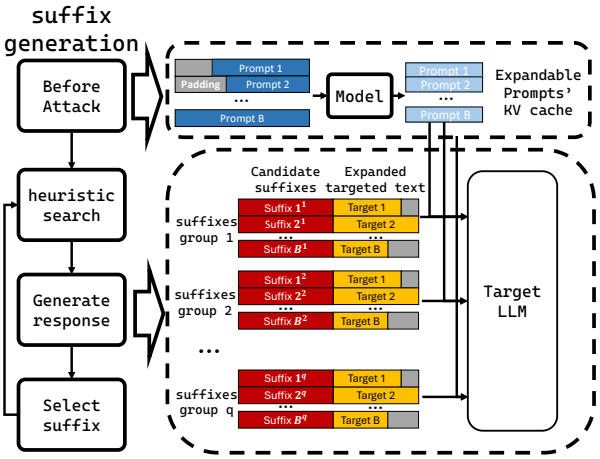

Figure 2. PSKV. By reusing the KV cache computed from the prefix, we only need to store one copy of the prompt for q new suffixes, rather than q copies as required by traditional methods.

4.3. Suffix-Centric Alignment for Batched Multi-Instruction Attack

In real-world benchmarking, evaluating robust accuracy requires optimizing suffixes for a batch of B distinct instruction-response pairs $\{(x_i^{(h)}, y_i^{(h)})\}_{i=1}^{B}$. A practical challenge arises from the variable token lengths of these pairs, which typically leads to ragged tensors that hinder efficient parallelization. To resolve this and maximize the utility of PSKV, we implement a **Suffix-Centric Alignment** strategy.

As illustrated in Figure 3, we normalize the data layout by left-padding all instructions $x_i^{(h)}$ and right-padding all target responses $y_i^{(h)}$. This alignment ensures that all generated suffixes start at the same column index within the batch tensor. Consequently, despite the variable input lengths, the suffixes form a single, dense token matrix of shape $(B \cdot K \cdot q) \times N_s$.

This design yields two critical benefits. The dense matrix representation allows the massive number of candidate suffixes ($B \cdot K \cdot q$) to be processed using highly optimized vectorized operations in standard frameworks (e.g., PyTorch), eliminating the computational overhead from sequential or irregular batching in loss evaluation and suffix retrieval. Meanwhile, the right-alignment of instructions aligns the memory layout of the static prefixes. This allows PSKV to pre-compute the KV cache once for the B instructions and broadcast it efficiently across the massive candidate

Table 2. Unified complexity analysis. E: epochs; N_{cand}: total candidates per iteration ($K \cdot q$ for single, $B \cdot K \cdot q$ for batch); N_p, N_s, N_t: lengths of prefix, suffix, and target. $L_{dec} = N_s + N_t$ denotes decoding length.

Scenario	Method	Computational Complexity	Peak Memory Complexity
Single-Instruction (1 Prompt)	No Accel.	$\mathcal{O}(E \cdot N_{cand} \cdot (N_p + L_{dec})^2)$	$\mathcal{O}(N_{cand} \cdot (N_p + L_{dec}))$
	Standard KV	$\mathcal{O}(N_p^2 + E \cdot N_{cand} \cdot L_{dec} \cdot (2N_p + L_{dec}))$	$\mathcal{O}(N_{cand} \cdot (\mathbf{N_p} + L_{dec}))$
	PSKV (Ours)	$\mathcal{O}(N_p^2 + E \cdot N_{cand} \cdot L_{dec} \cdot (2N_p + L_{dec}))$	$\mathcal{O}(\mathbf{1} \cdot N_p + N_{cand} \cdot L_{dec})$
Batched Multi-Instruction (B Prompts)	No Accel.	$\mathcal{O}(E \cdot N_{cand} \cdot (N_{p,\text{avg}} + L_{dec})^2)$	$\mathcal{O}(N_{cand} \cdot (N_{p,\text{avg}} + L_{dec}))$
	Standard KV	$\mathcal{O}(B \cdot N_p^2 + E \cdot N_{cand} \cdot L_{dec} \cdot (2N_p + L_{dec}))$	$\mathcal{O}(N_{cand} \cdot (\mathbf{N_{p,\text{max}}} + L_{dec}))$
	PSKV (Ours)	$\mathcal{O}(B \cdot N_p^2 + E \cdot N_{cand} \cdot L_{dec} \cdot (2N_p + L_{dec}))$	$\mathcal{O}(\mathbf{B} \cdot N_{p,\text{max}} + N_{cand} \cdot L_{dec})$

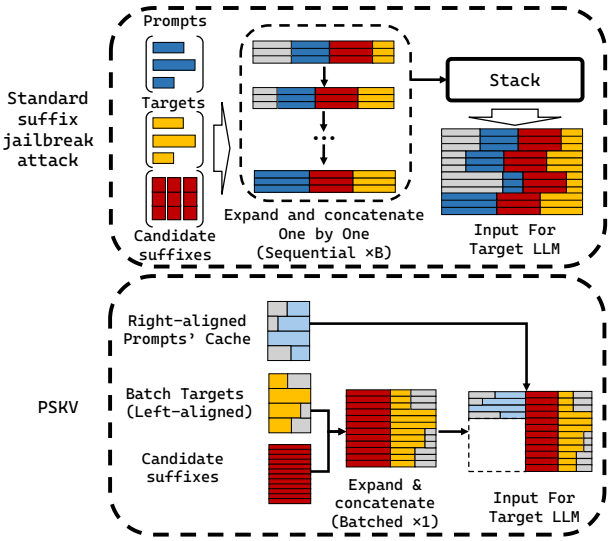

Figure 3. Alignment strategies for batched suffix attacks. The standard method's misaligned structure prevents prompt KV cache reuse. PSKV utilizes a suffix-centric alignment, enables the prompt's KV cache to be shared across all candidates and allows all components to be processed in a single, highly efficient tensor operation.

dimension, as analyzed in the complexity section below.

Complexity analysis (batch).

We analyze the complexity for a batch of B distinct prompts, generating a total of $N_{cand} = B \cdot K \cdot q$ candidates per iteration (detailed proofs in Appendix B.2). As summarized in Table 2, the **Standard KV cache** substantially reduces computational costs compared to the unaccelerated baseline, but it requires physically duplicating the prefix KV states for every candidate to match the input batch dimension, resulting in a prefix memory footprint of $\mathcal{O}(N_{cand} \cdot N_{p,\text{max}})$.

In contrast, by sharing the KV cache across all suffixes targeting the same prompt, PSKV reduces the prefix memory term to $\mathcal{O}(B \cdot N_{p,\text{max}})$. This term scales with the number of prompts B, not the much larger total candidate count N_{cand} (since typically $K \cdot q \gg 1$). This optimization effectively removes the memory barrier, making large-scale parallel attacks practical.

5. Experiments

In this section, we empirically explore how our proposed PSKV method can reduce the computation and memory costs of suffix jailbreakings while preserving their attack effectiveness.

5.1. Experimental Setup

Safety dataset. We use the first 50 examples from the test set of **Harmbench** (Mazeika et al., 2024) to form the safety dataset for the suffix jailbreaking evaluations. Each sample from this safety dataset consists of a harmful instruction and a targeted harmful response.

Targeted Models. We empirically validated PSKV using six suffix attacks on five target LLMs, which are: **Vicuna-7B-v1.5** (Chiang et al., 2023), **Llama-2-7B-Chat** (Touvron et al., 2023), **Llama-3-8B-Instruct** (Grattafiori et al., 2024), **Mistral-7B-Instruct-v0.3** (Jiang et al., 2023), and **Qwen2.5-7B-Instruct** (Yang et al., 2025).

Suffix jailbreak attacks. We benchmark how our PSKV can improve the efficiency of conducting six common suffix jailbreak attacks from two families. Specifically, **GCG** (Zou et al., 2023), **GCQ** (Hayase et al., 2024), **Zhu's AutoDAN** (Zhu et al., 2023), and **BEAST** (Sadasivan et al., 2024) are from the family of optimization-based attacks, which aim to search for jailbreak suffixes using various discrete optimization techniques. **AmpleGCG** (Liao & Sun, 2024) and **AdvPrompter** (Paulus et al., 2024) are from the family of model-based attacks, which aim to train LLM-based generators to directly generate jailbreak suffixes for given harmful instructions. Please refer to Appendix A for detailed settings of all other hyperparameters of all attacks. All of the methods were benchmarked on 2 80GB A100 GPUs.

Evaluation metrics. For the attack performance evaluation, we report the **Attack Success Rate (ASR)**, defined as

Table 3. Attack Success Rates (ASRs) (%) of suffix jailbreak attacks with/without PSKV. A high ASR indicate a strong jailbreak attack performance.

Model	Acceleration	GCG	GCQ	Zhu's AutoDAN	BEAST	AmpleGCG	AdvPrompter
Vicuna-7B	None	96	86	54	92	94	98
	PSKV (Ours)	94	96	54	100	96	98
Llama-2-7B	None	70	18	26	36	96	96
	PSKV (Ours)	56	16	28	46	88	96
Llama-3-8B	None	58	24	32	38	72	72
	PSKV (Ours)	62	26	30	54	76	82
Mistral-7B	None	80	72	86	86	86	94
	PSKV (Ours)	78	76	84	86	92	92
Qwen-2.5-7B	None	88	84	4	90	84	96
	PSKV (Ours)	88	92	6	86	88	94

the percentage of successful jailbreak attacks on the safety dataset. Please refer to App. A for details. For attack efficiency evaluation, we adopt memory consumption in gigabytes (GB) and execution time in minutes as the metrics.

Baseline acceleration methods. Our experiments evaluate the effectiveness of our proposed *PSKV* by comparing it against two alternatives: a baseline implementation where **no specific acceleration method is deployed** (which we denote as **None**), and an implementation using a **Standard KV Cache**.

None represents an unaccelerated, albeit batched, approach. In each iteration, it constructs $K \cdot q$ full candidate prompts by concatenating the prefix $x^{(h)}$ with each suffix. It then performs a single, large batched forward pass on these $K \cdot q$ sequences. This method redundantly calculates the prefix KV vectors for every candidate in every iteration. **Standard KV Cache** implements the intuitive optimization of pre-computing the prefix cache. It first performs a single forward pass on the prefix $x^{(h)}$ to obtain its KV cache. Then, to evaluate the $K \cdot q$ suffix candidates in a parallel batch, it physically replicates this prefix cache $K \cdot q$ times in GPU memory to match the batch dimension of the suffixes.

5.2. Results

In this section, we study the effectiveness of PSKV in accelerating suffix jailbreak attacks from different aspects.

PSKV does not affect the attack performance. According to the design of PSKV described in Section 4, PSKV is a performance-neutral acceleration method that does not affect the attack performance of accelerated suffix jailbreak attacks. This is because PSKV only removes redundant key-value vectors and does not change the algorithm itself or the computational accuracy. Nevertheless, we conduct experiments here to empirically justify this. We collect and present the ASRs of the jailbreak attacks that without any accelerations and those that adopt PSKV in Table 3.

From the table, we observe minor fluctuations in ASR between the baseline and PSKV-accelerated attacks. These variations stem not from PSKV altering the core attack logic but from the inherent stochasticity (e.g, heuristic search algorithms or sampling-based generation of multiple responses for ASR computation) in the end-to-end evaluation pipeline. Overall, the attack performance remains broadly consistent across all cases, demonstrating that PSKV effectively accelerates the attacks without systematically harming their original effectiveness.

PSKV considerably accelerates suffix jailbreaking. Table 4 details the time consumption of varying acceleration strategies.First, for optimization-based attacks that perform only one or two forward passes per candidate suffix step (including GCG, GCQ, BEAST, and AmpleGCG), our PSKV (PSKV) consistently achieves a speed-up ratio of approximately $1.4\times$ to $1.6\times$ compared to the no-acceleration baseline.Notably, this acceleration becomes even more pronounced for Zhu's AutoDAN, where the speed-up ratio rises to a range of $\sim 1.8\times$ to $1.95\times$ (e.g., reducing time from 88 to 45 minutes on Llama-3-8B). This performance gain is attributed to the mechanism of KV cache acceleration: it reduces computational redundancy by storing states. Since Zhu's AutoDAN requires substantially more inference steps per iteration (due to its hierarchical genetic algorithm), there is a larger proportion of redundant prefix computation for PSKV to eliminate.

Conversely, for AdvPrompter, the speed-up ratio decreases to approximately $1.3\times$. This is because AdvPrompter involves synchronous reinforcement learning; the total time includes both inference and generator training. While PSKV accelerates the inference phase, the training overhead remains constant, diluting the overall proportion of time saved. Furthermore, compared to the Standard KV Cache, our method maintains a comparable speed without incurring time penalties (in some cases, such as Llama-3 AdvPrompter, it is even faster). This indicates that the overhead caused by our temporary KV tensor expansion is negligible relative to the total inference time.

Table 4. Time usage (min) for (1) optimization-based suffix synthesis and (2) model-based training sample generation. Parentheses (×) indicate speed-up factors vs. the unaccelerated baseline. **OOM**: Out Of Memory.

Model	Acceleration	Optimization-based Attacks				Model-based Attacks	
		GCG	GCQ	Zhu's AutoDAN	BEAST	AmpleGCG	AdvPrompter
Vicuna-7B	None	228 (× 1.00)	247 (× 1.00)	88 (× 1.00)	23 (× 1.00)	349 (× 1.00)	462 (× 1.00)
	Standard KV Cache	147 (× 1.55)	OOM	OOM	OOM	239 (× 1.46)	160 (× 2.89)
	PSKV (Ours)	146 (× 1.56)	148 (× 1.67)	50 (× 1.76)	15 (× 1.53)	253 (× 1.38)	199 (× 2.32)
Llama-2-7B	None	221 (× 1.00)	210 (× 1.00)	86 (× 1.00)	26 (× 1.00)	349 (× 1.00)	513 (× 1.00)
	Standard KV Cache	142 (× 1.56)	OOM	OOM	OOM	242 (× 1.44)	344 (× 1.49)
	PSKV (Ours)	142 (× 1.56)	145 (× 1.45)	48 (× 1.79)	14 (× 1.86)	227 (× 1.54)	275 (× 1.87)
Llama-3-8B	None	288 (× 1.00)	221 (× 1.00)	88 (× 1.00)	24 (× 1.00)	329 (× 1.00)	577 (× 1.00)
	Standard KV Cache	151 (× 1.91)	141 (× 1.57)	48 (× 1.83)	OOM	240 (× 1.37)	282 (× 2.05)
	PSKV (Ours)	141 (× 2.04)	138 (× 1.60)	45 (× 1.96)	14 (× 1.71)	242 (× 1.36)	307 (× 1.88)
Mistral-7B	None	211 (× 1.00)	203 (× 1.00)	81 (× 1.00)	25 (× 1.00)	320 (× 1.00)	96 (× 1.00)
	Standard KV Cache	138 (× 1.53)	126 (× 1.61)	40 (× 2.03)	OOM	229 (× 1.40)	80 (× 1.20)
	PSKV (Ours)	131 (× 1.61)	122 (× 1.66)	40 (× 2.03)	14 (× 1.79)	230 (× 1.39)	74 (× 1.30)
Qwen2.5-7B	None	250 (× 1.00)	242 (× 1.00)	84 (× 1.00)	26 (× 1.00)	295 (× 1.00)	102 (× 1.00)
	Standard KV Cache	134 (× 1.87)	143 (× 1.69)	44 (× 1.91)	OOM	223 (× 1.32)	81 (× 1.26)
	PSKV (Ours)	135 (× 1.85)	128 (× 1.89)	42 (× 2.00)	15 (× 1.73)	237 (× 1.24)	76 (× 1.34)

Finally, observing across different target models, the variance in time usage is largely within the range of random fluctuations.[1] Since the sequence lengths are relatively short (avg. ∼190 tokens), the computational bottleneck lies in computation rather than throughput limitations imposed by model architecture.

Memory Efficiency Analysis of PSKV. Table 5 reveals that PSKV (PSKV) consistently achieves superior memory efficiency, often surpassing even the "None" baseline by avoiding the storage of expanded input tensors required for batch processing. Unlike Standard KV Cache, which suffers from memory explosion under large beam widths by maintaining a fully expanded state, PSKV leverages layer-wise inference to dynamically expand and immediately discard the KV view for the current layer, keeping the resident footprint minimal.

Memory savings vary by attack dynamics: for gradient-based attacks (GCG) where the KV cache dominates memory usage, PSKV achieves high compression; conversely, while massive intermediate tensors in beam-search methods (GCQ, BEAST) lower the relative reduction ratio, PSKV remains vital for preventing the OOM errors frequent in Standard Cache. Similarly, PSKV mitigates cumulative cache growth in AutoDAN, though in dual-model setups like AdvPrompter, the increased non-KV memory burden can marginally offset these savings.

Architecturally, the efficiency gap is most pronounced with Multi-Head Attention (MHA) models (Vicuna-7B, Llama-2-7B) where PSKV eliminates the redundancy of storing unique pairs for every head.[2] In contrast, models utilizing Grouped-Query Attention (GQA) like Llama-3, Mistral,[3] and Qwen2.5 inherently reduce the baseline KV size, lowering PSKV's theoretical compression ceiling. Furthermore, the larger vocabularies in these models generate massive intermediate tensors (logits, IDs) that further dilute the KV cache's relative memory proportion, rendering relative savings less apparent.

See App. D for a more detailed analysis of memory consumption.

Scalability Analysis. Table 6 highlights PSKV's efficient memory scalability. Remarkably, the total peak memory consumption remains virtually constant despite a 4× increase in search width (e.g., GCG stabilizes at ≈40.8 GB across all widths). This stability arises because the peak memory footprint is dominated by substantial intermediate tensors required for gradient computation and loss computation, rather than the inference cache itself. Since PSKV enables efficient prefix sharing, the marginal memory cost of expanding the search width is negligible and remains entirely masked beneath the high-water mark set by these non-cache components. Consequently, the amortized memory cost per candidate drops inversely with the width, demonstrating that PSKV enables high-width parallel search without raising the hardware memory barrier.

Comparisons with State-of-the-Art Inference Systems. We include vLLM (Kwon et al., 2023b) and SGLang (Zheng et al., 2024) in our evaluation to benchmark our method against comprehensively optimized inference systems used

[1] We also observe that AdvPrompter requires significantly more time on Llama-series models compared to Mistral and Qwen. For explanations, please refer to App. C.

[2] See App. D for architectural details.

[3] Mistral's sliding window does not impact these results as inputs fit within the window.

Table 5. Memory consumption (GB) for (1) optimization-based suffix synthesis and (2) model-based training sample generation. **OOM**: Out Of Memory. (×): Reduction factor vs. Standard KV Cache; (−) denotes baseline OOM.

Model	Acceleration	Optimization-based Attacks				Model-based Attacks	
		GCG	GCQ	AutoDAN	BEAST	AmpleGCG	AdvPrompter
Vicuna-7B	None	30.02 (× 3.35)	66.63 (-)	90.23 (-)	144.42 (-)	145.51 (× 1.02)	124.57 (× 1.25)
	Standard KV Cache	100.47 (× 1.00)	OOM	OOM	OOM	147.79 (× 1.00)	155.46 (× 1.00)
	PSKV (Ours)	27.32 (× 3.68)	55.75 (-)	52.71 (-)	92.02 (-)	62.50 (× 2.36)	66.88 (× 2.32)
Llama-2-7B	None	35.25 (× 2.84)	62.82 (-)	102.20 (-)	111.97 (-)	145.23 (× 1.02)	72.78 (× 2.09)
	Standard KV Cache	100.10 (× 1.00)	OOM	OOM	OOM	147.95 (× 1.00)	151.98 (× 1.00)
	PSKV (Ours)	29.57 (× 3.39)	50.23 (-)	51.35 (-)	81.20 (-)	58.02 (× 2.55)	63.37 (× 2.40)
Llama-3-8B	None	55.70 (× 1.05)	118.07 (× 1.06)	123.73 (× 1.10)	142.06 (-)	128.05 (× 0.88)	117.24 (× 1.27)
	Standard KV Cache	58.28 (× 1.00)	125.42 (× 1.00)	136.41 (× 1.00)	OOM	112.21 (× 1.00)	148.65 (× 1.00)
	PSKV (Ours)	40.84 (× 1.43)	85.07 (× 1.47)	77.25 (× 1.77)	110.29 (-)	90.74 (× 1.24)	126.27 (× 1.18)
Mistral-7B	None	34.36 (× 1.33)	53.66 (× 1.56)	84.78 (× 1.35)	144.42 (-)	86.27 (× 0.93)	53.51 (× 1.69)
	Standard KV Cache	45.53 (× 1.00)	83.52 (× 1.00)	114.41 (× 1.00)	OOM	79.84 (× 1.00)	90.22 (× 1.00)
	PSKV (Ours)	28.03 (× 1.62)	43.64 (× 1.91)	44.18 (× 2.59)	119.52 (-)	58.21 (× 1.37)	65.75 (× 1.37)
Qwen2.5-7B	None	66.03 (× 0.81)	121.73 (× 0.94)	140.91 (× 0.89)	139.30 (-)	120.74 (× 0.93)	126.76 (× 1.15)
	Standard KV Cache	53.65 (× 1.00)	114.94 (× 1.00)	125.47 (× 1.00)	OOM	111.94 (× 1.00)	145.98 (× 1.00)
	PSKV (Ours)	41.76 (× 1.28)	94.17 (× 1.22)	82.23 (× 1.53)	114.75 (-)	94.43 (× 1.19)	138.28 (× 1.06)

Table 6. Scalability of iterative/search-based attacks with PSKV on HarmBench using a Llama-3-8B target.

Attack	Width	Total Cost		Cost/width		ASR (%)
		Time (min)	Mem. (GB)	Time (min)	Mem. (GB)	
GCG	16	40	40.83	2.50	2.55	50
	32	73	40.83	2.28	1.28	54
	64	141	40.84	2.20	0.64	62
GCQ	16	34	85.06	2.13	5.32	26
	32	70	85.07	2.19	2.66	26
	64	138	85.07	2.16	1.33	26
BEAST	5	5	110.27	1.00	22.05	40
	10	9	110.29	0.90	11.03	50
	15	14	110.29	0.93	7.35	54

in production. While these frameworks utilize advanced memory management techniques like PagedAttention, they lack the gradient support required for white-box optimization(e.g., GCG, AdvPrompter). However, for the gradient-free **BEAST** attack, they serve as critical baselines. By comparing against these highly specialized engines, we demonstrate the computational superiority and efficiency of our implementation.

Table 7 benchmarks the performance of the BEAST attack on specialized inference frameworks. Although vLLM and SGLang exhibit impressive efficiency through their full-stack optimized kernels, their design explicitly sacrifices gradient computation, making them unsuitable for white-box optimization tasks. PSKV-accelerated BEAST (see Tables 4 and 5) retains full automatic differentiation support within a native PyTorch environment, yielding performance that is only slightly inferior to—or in some configurations, even comparable to—these two industry-standard engines. Furthermore, while vLLM and SGLang are restricted to gradient-free attacks, PSKV serves as a universal plug-and-play accelerator across the entire spectrum of suffix jailbreak methodologies, bridging the gap between high-performance inference and flexible adversarial optimization.

Table 7. Efficiency of gradient-free BEAST attack on specialized inference frameworks (SGLang and vLLM).

framework	Cost	model				
		vicuna	llama2	llama3	mistral	Qwen2.5
sglang	Time	8:49	9:25	6:22	7:00	6:12
	Mem.	13.42	12.87	14.96	13.50	14.23
vLLM	Time	16:02	16:50	17:58	16:27	15:51
	Mem.	12.55	12.57	14.96	12.73	13.52

6. Conclusion

The effectiveness of suffix jailbreak attacks is fundamentally tied to heuristic searches that require a vast number of resource-intensive LLM inferences, creating a significant barrier to efficient security evaluation. Our work is motivated by the key observation : for a single harmful prompt, numerous candidate suffixes are generated, yet standard methods create multiple redundant copies of the prompt's computational state, leading to massive memory waste. To address this inefficiency, we propose the *Prefix-Shared KV Cache (PSKV)* framework. The core principle of PSKV is to maintain only a single, shared, and expandable KV cache for each harmful instruction prefix, which is then efficiently reused for all its corresponding suffix candidates. To enable this sharing mechanism at scale for batches of diverse prompts, we also introduce a suffix-centric alignment strategy. We empirically validated PSKV across six suffix attacks on five LLMs, demonstrating an average reduction of 40% in time and 50% in memory compared to cache-optimized baselines, while preserving the original attack success rate.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

Chao, P., Robey, A., Dobriban, E., Hassani, H., Pappas, G. J., and Wong, E. Jailbreaking black box large language models in twenty queries, 2024. URL https://arxiv.org/abs/2310.08419.

Chiang, W.-L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Gonzalez, J. E., Stoica, I., and Xing, E. P. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality. LMSYS Org Blog, mar 2023. URL https://lmsys.org/blog/2023-03-31-vicuna/. Accessed: 2025-09-14.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv e-prints*, pp. arXiv–2407, 2024.

Hayase, J., Borevković, E., Carlini, N., Tramèr, F., and Nasr, M. Query-based adversarial prompt generation. *Advances in Neural Information Processing Systems*, 37: 128260–128279, 2024.

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mistral 7b, 2023. URL https://arxiv.org/abs/2310.06825.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023a.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention, 2023b. URL https://arxiv.org/abs/2309.06180.

Liao, Z. and Sun, H. Amplegcg: Learning a universal and transferable generative model of adversarial suffixes for jailbreaking both open and closed llms. *arXiv preprint arXiv:2404.07921*, 2024.

Mazeika, M., Phan, L., Yin, X., Zou, A., Wang, Z., Mu, N., Sakhaee, E., Li, N., Basart, S., Li, B., Forsyth, D., and Hendrycks, D. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal, 2024. URL https://arxiv.org/abs/2402.04249.

Paulus, A., Zharmagambetov, A., Guo, C., Amos, B., and Tian, Y. Advprompter: Fast adaptive adversarial prompting for llms. *arXiv preprint arXiv:2404.16873*, 2024.

Sadasivan, V. S., Saha, S., Sriramanan, G., Kattakinda, P., Chegini, A., and Feizi, S. Fast adversarial attacks on language models in one gpu minute. In *International Conference on Machine Learning*, 2024.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need, 2023. URL https://arxiv.org/abs/1706.03762.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Ye, L., Tao, Z., Huang, Y., and Li, Y. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition. *arXiv preprint arXiv:2402.15220*, 2024.

Zheng, L., Yin, L., Xie, Z., Sun, C. L., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.

Zhu, S., Zhang, R., An, B., Wu, G., Barrow, J., Wang, Z., Huang, F., Nenkova, A., and Sun, T. Autodan: Interpretable gradient-based adversarial attacks on large language models, 2023. URL https://arxiv.org/abs/2310.15140.

Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z., and Fredrikson, M. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

A. Implementation Details of Jailbreak Attacks

Attack Methods. Our experiments adopt six token-level jailbreak attacks, including four optimization-based methods: GCG (Zou et al., 2023), BEAST (Sadasivan et al., 2024), GCQ (Hayase et al., 2024), and Zhu's AutoDAN (Zhu et al., 2023), as well as two model-based attacks: AmpleGCG (Liao & Sun, 2024) and AdvPrompter (Paulus et al., 2024). We re-implemented all attacks to apply PSKV and enable efficient batched operations. We re-implemented all six evaluated suffix jailbreak attacks ourselves to make them compatible with our PSKV method. We fixed jailbreak suffixes' length to 20 in every experiment. The hyperparameters for each attack are configured as follows:

- **GCG:** Following Algorithm 1 in Zou et al. (2023) and its official repository, we set: iterations = 500, top-k = 256, batch size (search width) = 64, prompt batch size = 16, and search width batch size = 16.
- **GCQ:** Following Algorithm 1 in Hayase et al. (2024), we set: iterations = 500, top-k = 256, proxy batch size and query batch size = 15, buffer size (search width) = 64, and batch size for prompts and buffer = 16.
- **BEAST:** Following Algorithm 1 in Sadasivan et al. (2024) and its official repository, we set: both beam-search parameters = 15, and beam-search batch size = 15.
- **Zhu's AutoDAN:** Following Algorithms 1 and 2 in Zhu et al. (2023), we set: iterations = 3, objective weights = (10, 100), top-B = 256, and temperature = 2.
- **AmpleGCG:** Following Algorithm 1 in Liao & Sun (2024) and its official repository, we set: steps = 500, batch size (search width) = 64, and prompt batch size = 16.
- **AdvPrompter:** Following Algorithm 1 in Paulus et al. (2024) and its official repository, we set: epochs = 10, candidate set size = 256, prompt batch size = 8, temperature = 0.6, penalty parameter = 1.2, beam set size = 64, and beam number = 4.

Attack Success Rate(ASR). For each sample from the safety dataset, we perform a jailbreak attack to synthesize a jailbreak suffix and use it to induce the targeted LLMs to generate 10 responses. An LLM-based judger [4] from Mazeika et al. (2024) is then used to evaluate whether a response contains harmful information. An attack is determined to be successful if any of the 10 responses is determined to be harmful.

B. Proof: time and memory complex

B.1. Complexity Analysis for Single Prompt

In this section, we provide the derivations for the computational (C) and memory (M) complexities presented in Section 4.2. We let E be the number of synthesis iterations; N_p, N_s, N_t be the token lengths of the prefix, suffix, and target, respectively; and $K \cdot q$ be the number of candidates evaluated per iteration. Our analysis abstracts away model-specific constants (e.g., hidden dimension, number of layers) as they are common factors across all methods. The core assumptions are that the attention mechanism has a computational cost of $\mathcal{O}(L^2)$ for a sequence of length L, and the corresponding KV cache has a memory footprint of $\mathcal{O}(L)$.

B.1.1. Method without Acceleration

Computational Complexity. This method recomputes the entire sequence of length $N_p + N_s + N_t$ for each of the $K \cdot q$ candidates in every one of the E iterations. The total complexity is the product of the number of operations and the cost per operation:

$$\begin{aligned} C_{\text{NoAccel}} &= E \cdot (K \cdot q) \cdot \mathcal{O}((N_p + N_s + N_t)^2) \\ &= \mathcal{O}(E \cdot (K \cdot q) \cdot (N_p + N_s + N_t)^2). \end{aligned} \tag{4}$$

[4] https://huggingface.co/cais/HarmBench-Llama-2-13b-cls

Memory Complexity. The peak memory is determined by the batched inference step, which must hold the KV caches for all $K \cdot q$ candidates simultaneously. The memory for each cache is proportional to the full sequence length.

$$\begin{aligned} M_{\text{NoAccel}} &= (K \cdot q) \cdot \mathcal{O}(N_p + N_s + N_t) \\ &= \mathcal{O}((K \cdot q) \cdot (N_p + N_s + N_t)). \end{aligned} \tag{5}$$

B.1.2. STANDARD KV CACHE

Computational Complexity. This method involves a one-time prefix computation and an iterative evaluation step over E iterations.

- **Prefix Computation (One-time):** A single forward pass over the prefix of length N_p costs $\mathcal{O}(N_p^2)$.
- **Iterative Evaluation (per epoch):** For each of the $N_s + N_t$ tokens of the suffix and target, attention is computed over a context that grows from N_p to $N_p + N_s + N_t - 1$. The average context length for these tokens is approximately $N_p + (N_s + N_t)/2$. The computational cost for each of the $K \cdot q$ candidates is thus roughly proportional to the number of new tokens times twice the average context length, giving $\mathcal{O}((N_s + N_t) \cdot (2N_p + N_s + N_t))$.

The total complexity is the sum of the one-time cost and the total iterative cost:

$$\begin{aligned} C_{\text{Std}} &= \mathcal{O}(N_p^2) + E \cdot (K \cdot q) \cdot \mathcal{O}((N_s + N_t)(2N_p + N_s + N_t)) \\ &= \mathcal{O}(N_p^2 + E \cdot (K \cdot q) \cdot (N_s + N_t)(2N_p + N_s + N_t)). \end{aligned} \tag{6}$$

Memory Complexity. To achieve parallel computation, the pre-computed prefix cache of length N_p must be physically duplicated $K \cdot q$ times. The total memory is the sum of these duplicated prefix caches and the caches for the suffix and target segments.

$$\begin{aligned} M_{\text{Std}} &= (K \cdot q) \cdot \mathcal{O}(N_p) + (K \cdot q) \cdot \mathcal{O}(N_s + N_t) \\ &= \mathcal{O}((K \cdot q) \cdot (N_p + N_s + N_t)). \end{aligned} \tag{7}$$

B.1.3. PSKV (OURS)

Computational Complexity. The computational process of PSKV is identical to the standard KV cache approach, as it also performs a one-time prefix computation followed by iterative evaluation of the suffix and target segments.

$$C_{\text{Ours}} = C_{\text{Std}} = \mathcal{O}(N_p^2 + E \cdot (K \cdot q) \cdot (N_s + N_t)(2N_p + N_s + N_t)). \tag{8}$$

Memory Complexity. The key advantage of PSKV is that it avoids the physical duplication of the prefix cache. It stores the prefix cache of length N_p only once, alongside the caches for the $K \cdot q$ suffix and target segments.

$$\begin{aligned} M_{\text{Ours}} &= \mathcal{O}(N_p) + (K \cdot q) \cdot \mathcal{O}(N_s + N_t) \\ &= \mathcal{O}(N_p + (K \cdot q) \cdot (N_s + N_t)). \end{aligned} \tag{9}$$

B.2. Derivations for Batched Complexity Analysis

In this section, we provide the derivations for the complexities of batched suffix synthesis presented in Section 4.3. We let B be the number of prompts, E be the number of synthesis iterations, and $K \cdot q$ be the candidates per prompt per iteration. The total candidates per iteration is $B \cdot K \cdot q$. Let $N_{p,\text{max}}, N_s, N_{t,\text{max}}$ be the maximum token lengths of the aligned prefix, suffix, and target, and $N_{p,\text{avg}}, N_{t,\text{avg}}$ be the average lengths.

B.2.1. METHOD WITHOUT ACCELERATION

Computational Complexity. This method recomputes the full sequence for every candidate in every iteration. The total number of sequences processed is $E \cdot B \cdot K \cdot q$. The cost for each is quadratic to its length.

$$\begin{aligned} C_{\text{NoAccel}} &= E \cdot (B \cdot K \cdot q) \cdot \mathcal{O}((N_{p,\text{avg}} + N_s + N_{t,\text{avg}})^2) \\ &= \mathcal{O}(E \cdot (B \cdot K \cdot q) \cdot (N_{p,\text{avg}} + N_s + N_{t,\text{avg}})^2). \end{aligned} \tag{10}$$

Memory Complexity. The peak memory is determined by the batched inference step, which holds the KV caches for all $B \cdot K \cdot q$ candidates simultaneously.

$$\begin{aligned} M_{\text{NoAccel}} &= (B \cdot K \cdot q) \cdot \mathcal{O}(N_{p,\text{avg}} + N_s + N_{t,\text{avg}}) \\ &= \mathcal{O}((B \cdot K \cdot q) \cdot (N_{p,\text{avg}} + N_s + N_{t,\text{avg}})). \end{aligned} \tag{11}$$

B.2.2. STANDARD KV CACHE

Computational Complexity. This method pre-computes the cache for the B prefixes once, then iteratively evaluates the suffixes and targets.

- **Prefix Computation (One-time):** A batched pass over B prefixes of max length $N_{p,\text{max}}$ costs $\mathcal{O}(B \cdot N_{p,\text{max}}^2)$.
- **Iterative Evaluation (per epoch):** For each of the $B \cdot K \cdot q$ candidates, processing the $N_s + N_{t,\text{max}}$ new tokens costs approximately $\mathcal{O}((N_s + N_{t,\text{max}}) \cdot (2N_{p,\text{max}} + N_s + N_{t,\text{max}}))$.

The total complexity over E iterations is the sum of the one-time cost and the total iterative cost:

$$C_{\text{Std}} = \mathcal{O}(B \cdot N_{p,\text{max}}^2 + E \cdot (B \cdot K \cdot q) \cdot (N_s + N_{t,\text{max}})(2N_{p,\text{max}} + N_s + N_{t,\text{max}})). \tag{12}$$

Memory Complexity. To process in a single batch, the B prefix caches must be duplicated $K \cdot q$ times each. The total memory is the sum of these duplicated prefix caches and the caches for the suffix/target segments.

$$\begin{aligned} M_{\text{Std}} &= (B \cdot K \cdot q) \cdot \mathcal{O}(N_{p,\text{max}}) + (B \cdot K \cdot q) \cdot \mathcal{O}(N_s + N_{t,\text{max}}) \\ &= \mathcal{O}((B \cdot K \cdot q) \cdot (N_{p,\text{max}} + N_s + N_{t,\text{max}})). \end{aligned} \tag{13}$$

B.2.3. PSKV (OURS)

Computational Complexity. The computational process is identical to the standard KV cache approach.

$$C_{\text{Ours}} = C_{\text{Std}} = \mathcal{O}(B \cdot N_{p,\text{max}}^2 + E \cdot (B \cdot K \cdot q) \cdot (N_s + N_{t,\text{max}})(2N_{p,\text{max}} + N_s + N_{t,\text{max}})). \tag{14}$$

Memory Complexity. PSKV avoids physical duplication. It stores one cache for each of the B prefixes, and the caches for the $B \cdot K \cdot q$ suffix/target segments.

$$\begin{aligned} M_{\text{Ours}} &= B \cdot \mathcal{O}(N_{p,\text{max}}) + (B \cdot K \cdot q) \cdot \mathcal{O}(N_s + N_{t,\text{max}}) \\ &= \mathcal{O}(B \cdot N_{p,\text{max}} + (B \cdot K \cdot q) \cdot (N_s + N_{t,\text{max}})). \end{aligned} \tag{15}$$

C. Computational Efficiency Analysis of AdvPrompter

C.1. Observations

In our experimental evaluation of AdvPrompter (Table 4), we observed two notable phenomena:

1. **Severe performance degradation without KV caching:** All models exhibit substantially higher latency in the "None" (no acceleration) setting compared to cached configurations. For instance, Llama-2-7B increases from 275s (Standard KV Cache) to over 500s (None), representing an approximately $2\times$ slowdown.
2. **Model-specific latency disparities:** Among models of comparable size, Llama-2-7B and Llama-3-8B consistently exhibit higher execution times than Mistral-7B and Qwen2.5-7B across all acceleration settings. Even with our PSKV, Llama-2 (250s) and Llama-3 (282s) remain slower than Mistral (80s) and Qwen (76s).

This section analyzes the algorithmic and architectural factors underlying these observations.

C.2. The Cost of Cache-less Decoding

The severe slowdown in the "None" setting stems from the fundamental inefficiency of autoregressive generation without KV caching.

Quadratic Total Complexity. With KV caching, generating token t requires computing attention only between the new query and cached keys/values, yielding $O(1)$ complexity per layer with respect to history length. Without caching, the model must recompute all previous hidden states, requiring $O(t)$ operations at step t. Summing over a sequence of length n, the total complexity degrades from $O(n)$ to $O(n^2)$.

Amplification by AdvPrompter's Search Strategy. Unlike gradient-based attacks (*e.g.*, GCG) that perform single forward-backward passes, AdvPrompter employs beam search with sampling during suffix generation. This expands the effective batch size multiplicatively (batch $\times$ beam $\times$ samples), causing the quadratic recomputation penalty to be incurred across many parallel sequences simultaneously. The resulting computational burden saturates GPU throughput and explains the disproportionate slowdown observed in the "None" setting.

C.3. Architectural Factors in Llama-Series Latency

While all models benefit from KV caching, Llama-2 and Llama-3 exhibit consistently higher latency than Mistral and Qwen. We attribute this to two architecture-specific factors.

Llama-2: Multi-Head Attention Overhead. Llama-2-7B employs Multi-Head Attention (MHA) with 32 key-value heads, whereas Mistral-7B and Qwen2.5-7B utilize Grouped-Query Attention (GQA) with 8 key-value heads. In cache-less decoding, the model must recompute K and V projections for the entire sequence history at each generation step. With $4\times$ more KV heads, Llama-2 incurs proportionally higher memory bandwidth requirements for loading projection weights and storing intermediate KV tensors. This memory-bound overhead is particularly pronounced when batch sizes are large, as in AdvPrompter's beam search.

Llama-3: Vocabulary Projection Overhead. Llama-3-8B adopts GQA, mitigating the MHA-related overhead. However, it employs a vocabulary of 128K tokens—approximately $4\times$ larger than Llama-2 and Mistral ($\sim$32K). AdvPrompter uniquely requires training a generator model, which involves computing full logit distributions over the vocabulary and backpropagating through the output projection layer. The computational cost of this $\mathbb{R}^d \to \mathbb{R}^{|V|}$ projection and its gradient scales linearly with vocabulary size, imposing a persistent overhead on Llama-3 that cannot be alleviated by KV caching alone.

The "None" baseline forces redundant recomputation across the entire sequence history at each decoding step, a cost amplified by AdvPrompter's beam search strategy. Llama-2 is further penalized by its MHA architecture, which demands higher memory bandwidth than GQA-based models. Llama-3, while adopting GQA, suffers from the computational burden of its large vocabulary during AdvPrompter's generator training phase. Our PSKV effectively eliminates redundant prefix computations, providing consistent speedups across all evaluated models.

D. Detailed analysis of memory overhead

To substantiate the memory efficiency analysis presented in the main text, we provide a detailed breakdown of the model architectures, the mathematical formulation of KV cache memory footprint, and the specific implementation differences between Standard KV Cache and our PSKV (PSKV).

D.1. Architectural Specifications of Target Models

The effectiveness of PSKV is intrinsically linked to the underlying attention mechanism and vocabulary size of the target Large Language Models (LLMs). Table 8 summarizes the critical architectural parameters for the models evaluated in this study.

Impact of Attention Mechanisms (MHA vs. GQA). The memory footprint of the KV cache is determined by the number of Key-Value heads.

Table 8. Architectural specifications of the evaluated models. **MHA**: Multi-Head Attention; **GQA**: Grouped-Query Attention; **SWA**: Sliding Window Attention.

Model	Attention Type	Vocab Size	Q Heads	KV Heads	KV Compression
Vicuna-1.5-7B	MHA	32,000	32	32	1:1
Llama-2-7B	MHA	32,000	32	32	1:1
Llama-3-8B	GQA	128,256	32	8	4:1
Mistral-7B-v0.1	GQA + SWA	32,000	32	8	4:1
Qwen2.5-7B	GQA	151,936	28	4	7:1

- **Multi-Head Attention (MHA):** Used by Llama-2 and Vicuna. Here, . Every query head requires a unique key-value pair, leading to maximum memory usage.
- **Grouped-Query Attention (GQA):** Used by Llama-3, Mistral, and Qwen. Here, . Multiple query heads share a single KV head (e.g., Qwen utilizes 28 query heads but only 4 KV heads). This architectural change physically reduces the KV cache tensor size by a factor of , which inherently lowers the memory ceiling and thus reduces the *relative* impact of PSKV compared to MHA models.

Impact of Vocabulary Size. The memory consumption during attack generation is not solely due to the KV cache but also intermediate activation tensors, specifically the logits tensor, which has a shape of . As shown in Table 8, Llama-3 and Qwen2.5 have vocabulary sizes () approx. to larger than Llama-2. This drastically increases the non-KV memory overhead (e.g., storing logits and gradients), further diluting the proportion of memory saved by PSKV.

Note on Mistral's Sliding Window. Mistral-7B employs Sliding Window Attention (SWA) with a window size of . While SWA theoretically caps the KV cache size to , our experiments utilize suffix attack sequences where the total length . Therefore, the cache behavior in our experiments effectively mirrors standard GQA without the sliding window truncation benefits.

D.2. Theoretical Memory Complexity: Standard vs. PSKV

We formalize the memory consumption to explain why Standard KV Cache results in OOM errors under beam search settings while PSKV remains efficient. Let be the batch size (or beam width/number of candidates), be the prefix length, be the generated tokens, be the hidden dimension, and be the number of layers.

Standard KV Cache (Eager Expansion). Standard implementations (e.g., Hugging Face default) eagerly expand the KV cache to match the batch dimension at initialization. The memory complexity for the cached prefix is:

$$M_{std} \propto N_{layer} \times \mathbf{B} \times L_{pre} \times H_{kv} \times D \tag{16}$$

Under optimization-based attacks (e.g., GCQ, BEAST), represents the number of candidates (often or). The term causes memory usage to scale linearly with the candidate count, leading to rapid exhaustion of GPU memory.

PSKV (Layer-wise Dynamic Expansion). Our PSKV implementation maintains a singleton copy of the prefix cache. During the forward pass of the -th layer, the cache is virtually broadcasted or temporarily expanded only for that specific layer's computation and immediately released. The resident memory footprint is:

$$M_{pskv} \propto N_{layer} \times \mathbf{1} \times L_{pre} \times H_{kv} \times D \tag{17}$$

The temporary expansion memory required during the forward pass is (for the current layer computation). This allows the memory consumption related to the prefix to remain independent of the attack batch size , providing the efficiency gains observed in the main results.

D.3. Attack-Specific Memory Overhead Details

Different attack algorithms impose different auxiliary memory costs that interact with the cache mechanism:

- **High-Width Search (GCQ, BEAST):** These algorithms generate a large number of candidate sequences per step. While PSKV compresses the prefix storage, the intermediate activations (e.g., attention scores, FFN intermediate states) still scale with . Thus, while PSKV prevents OOM from the *cache* perspective, the *activation* memory creates a new floor for memory usage.
- **Dual-Model Loading (AdvPrompter):** AdvPrompter requires the simultaneous residence of the Target LLM and the Attacker LLM (Prompter) in VRAM. This effectively halves the available memory for dynamic tensors. In this constrained environment, the static weight of the models dominates the memory profile, making the dynamic savings from PSKV less visually prominent in percentage terms compared to single-model attacks.